Chapter 17: AI Generated C#

In the previous two examples my goal was to get you thinking about the design process. How you work with the subject matter experts to learn about the current situation and then apply your design knowledge to convert that current state into something with real flexibility. It is all about the ability to evolve with the growth of your company and to quickly adapt to changes in the environment. If all goes well your design will make current processes more efficient and open new markets going forward.

In the world of C# programming the ability to quickly adapt to changes in the environment means recognizing that AI is something to take seriously. In just a few short years I expect AI will drastically change how we go about designing and programming an application.

As such, in this chapter we are going to look at some AI generated web pages and the code behind those pages.

Let's jump to the inside and examine AI generated C# code.

Before we get too deep into the weeds, I should tell you that I am a huge advocate of Microsoft's copilot. My PL/SQL and C# are very strong. My grammar is equally weak. The good news is that copilot always tells me when my grammar is broken beyond repair and even provides options for a fix with a simple right click. Cool, but what about C# applications? Are the AI tools equally advanced?

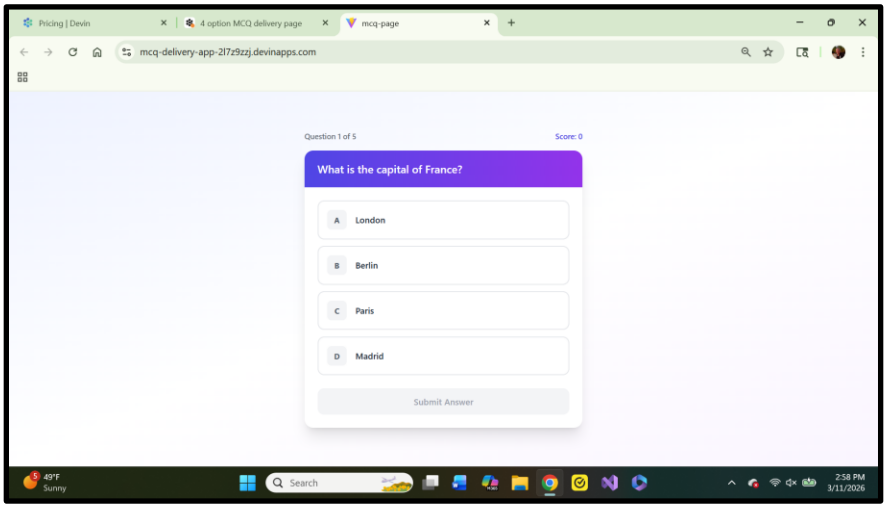
To answer this question, I of course went to the source of all knowledge, a.k.a. Google, and did a simple query "AI generated C#." Google came back with a cadre of options. I knew I didn't want to bother exploring Gemini, which is Visual Studio's built in AI tool, because it is just way too consistent, consistently wrong! As such, I went with the first commercial product listed by Google, Devin.

Devin is owned by Cognition AI out of Silicon Valley. Devin is recognized as the world's first fully autonomous AI software engineer. Cognition's CEO states that in 2026 Devin was writing north of 95% of Cognition's in-house code.

Cool, I get a free test drive and if I do decide I like it, I only pay $20/month. The interface was super easy to use. You simply type in your specification, and a few seconds later it spits out a URL for you to test what it had created.

My first specification was way too broad, I asked “please build me a multiple-choice display page.” The solution it came back with wasn’t what I expected, but it wasn’t too bad given my vague specification. (Figure 17.1) No next or previous buttons, only a stem, 4 options and a submit button. If you got the answer correct your selected option went green, if you got it wrong the option you selected went red and the correct option went green. In addition, my test environment only had a single question.

Figure 17.1: Specification Too Vague

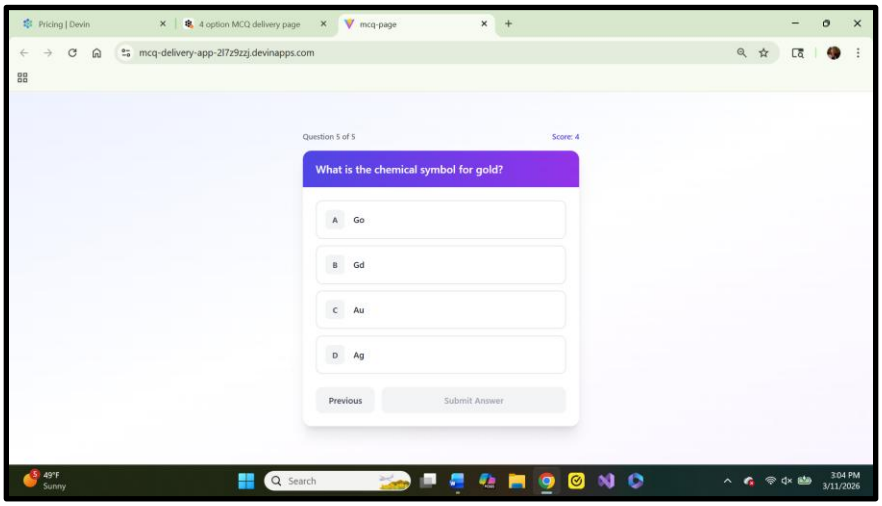


Okay, lesson learned. These tools are very literal. I needed to be more specific. My next specification was, “Okay, erase all that and build me a 4-option multiple-choice display screen with previous, next and submit buttons.” By the way, this tool’s ability to decode non-standard English was absolutely remarkable, much better than copilot or Gemini.

Devin came back just a couple of minutes later with 5 4-option questions and all three buttons just as I had described. On testing I still got the red/green feedback and all three buttons worked as expected. But again, this software is super literal. You have to be the smart adult in the room. I hadn’t told it to make the previous button invisible on the first question and the next button invisible on the last question.

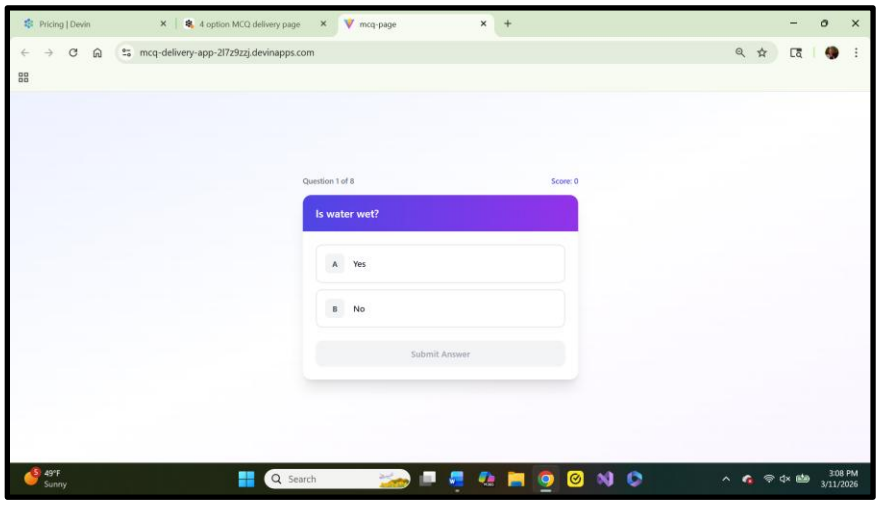
Devin’s next iteration of the page got it right. (Figure 17.2)

Figure 17.2: Buttons Correct



In my next specification I asked Devin to make my page support 2, 3, 4 or 5-option questions. Devin came back perfectly. Question 1 was a 2-option question. Question 2 was a 3-option question. Question 3 had 4 options and question 4 had 5 options. (Figure 17.3)

Figure 17.3: Varying Option Count



At this point I was pretty excited about Devin's ability to create a reasonable frontend, so it was time to look at the code behind this screen. As shown here (Figure 17.4 A-H), Devin's idea of C# doesn't look like any C# I have ever seen. In fact, when I put it into a C# Visual Studio project intellisense pretty much went crazy. Red underlines everywhere. Think Christmas tree.

Figure 17.4A: Code Behind Page

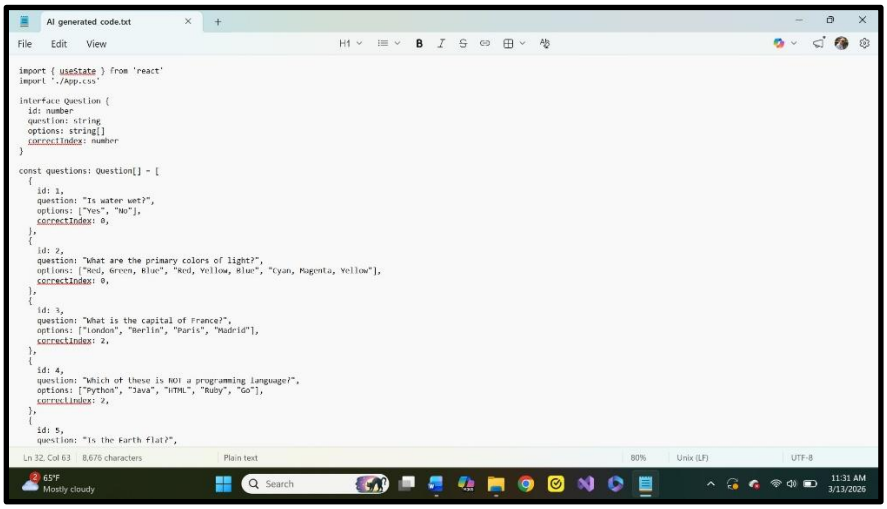


Figure 17.4B: Code Behind Page

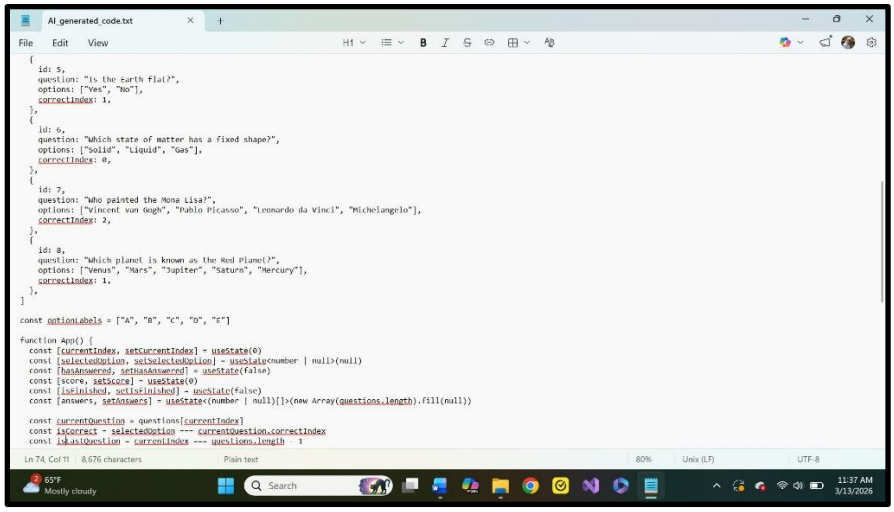


Figure 17.4C: Code Behind Page

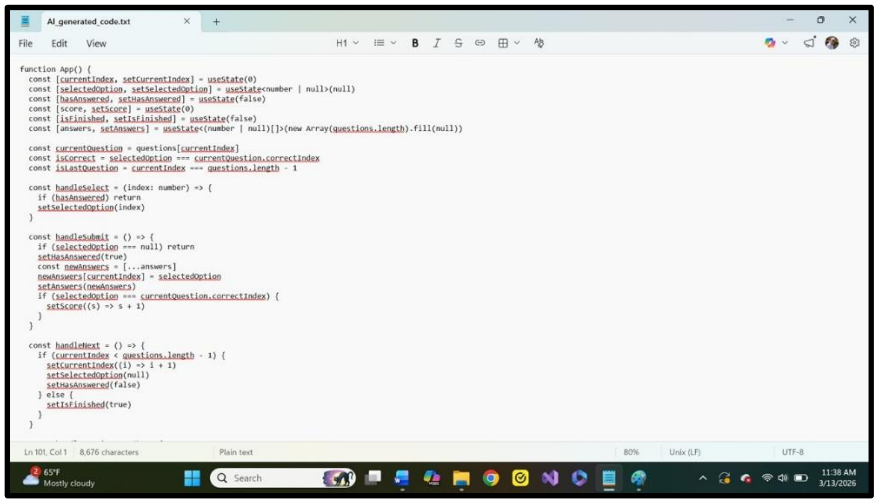


Figure 17.4D: Code Behind Page

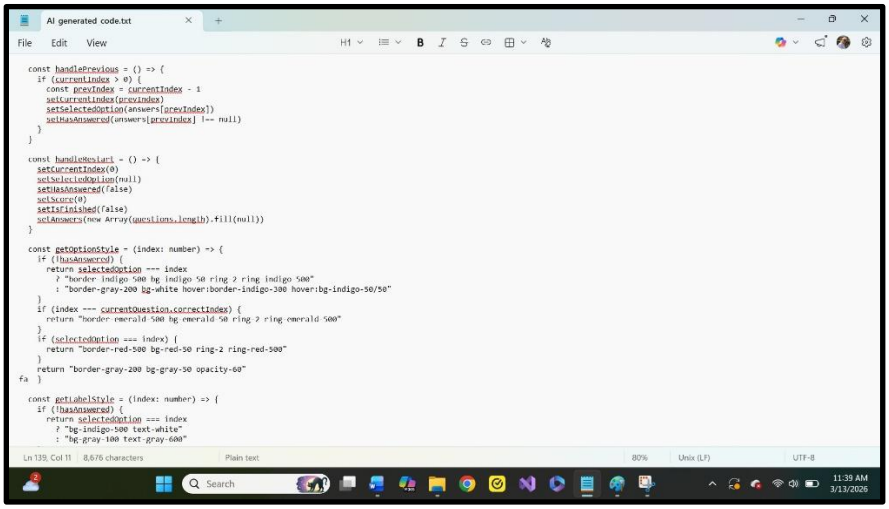


Figure 17.4E: Code Behind Page

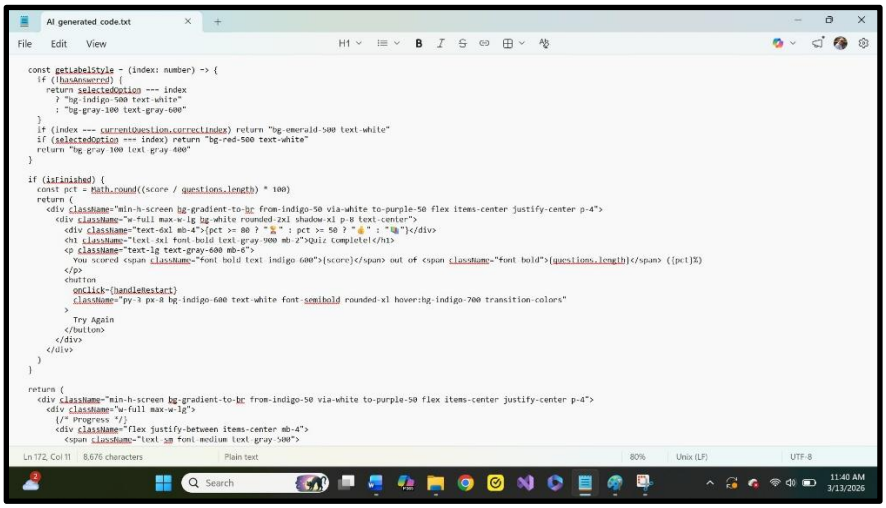


Figure 17.4F: Code Behind Page

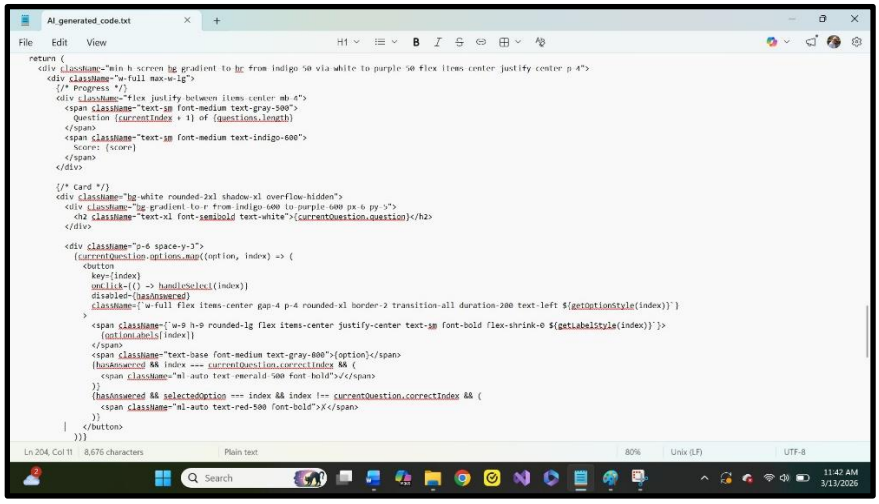


Figure 17.4G: Code Behind Page

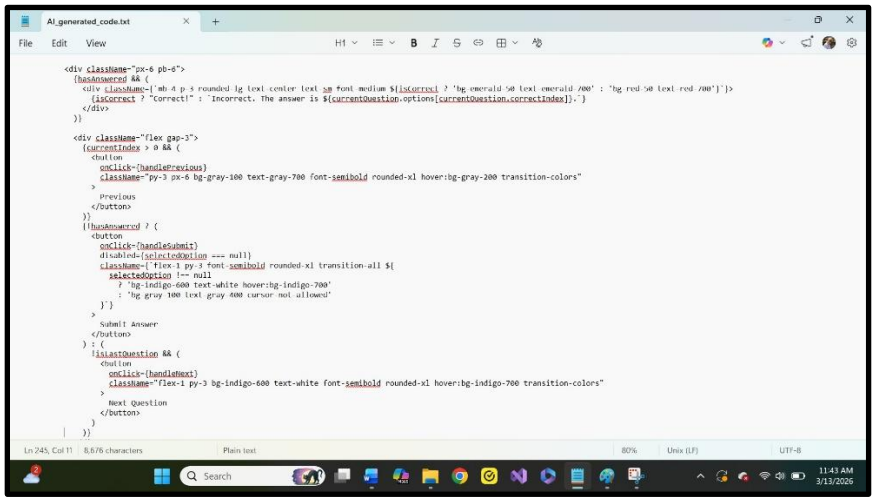
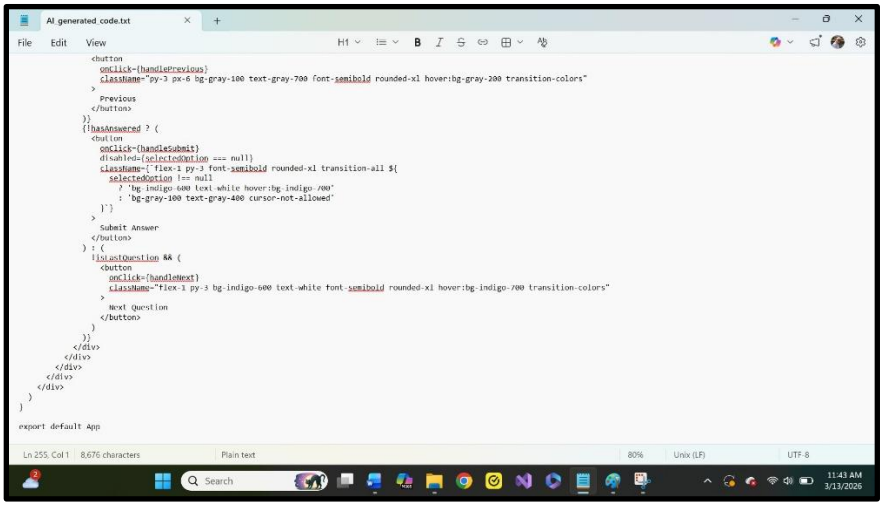


Figure 17.4H: Code Behind Page



So, what did we learn? We learned that Cognition AI tool Devin is a great way to put together a quick and dirty screen for interfacing with subject matter experts. I'd pay twenty bucks a month just for that. Unfortunately, the code behind those screens looks more like HTML and a formatting CSS than C#, totally unreadable. Net, Devin is not ready for primetime C# code.

Maybe it was just Devin which was inept, so I went to try my very favorite AI tool ChatGPT. Siri is dead and gone in my world. ChatGPT is the real deal for day-to-day work.

So, I gave ChatGPT the following instruction, "Please create a C# 4-option multiple-choice question display screen." As I'm sure you know, ChatGPT is super-fast. Whereas Devin took several minutes to generate a response, ChatGPT was back in just a couple of seconds. The response I got is shown here. (Figure 17.5)

Figure 17.5: ChatGPT Iteration 1

```
using System;

class Program
{
    static void Main()
    {
        string question = "Which planet is known as the Red Planet?";
        string[] options =
        {
            "A) Earth",
            "B) Mars",
            "C) Venus",
            "D) Jupiter"
        };

        char correctAnswer = 'B';

        Console.WriteLine(question);
        Console.WriteLine();

        foreach (var option in options)
        {
            Console.WriteLine(option);
        }

        Console.WriteLine();
        Console.Write("Enter your answer (A, B, C, or D): ");

        string input = Console.ReadLine()?.Trim().ToUpper();

        if (input == correctAnswer.ToString())
        {
            Console.WriteLine("Correct!");
        }
        else
        {
            Console.WriteLine($"Incorrect. The correct answer is {correctAnswer}.");
        }
    }
}
```

Hello, now that looks like C#. Granted it is sort of at the level we were writing 10-15 chapters ago, but I ran it and it compiled and tested perfectly. Cool, maybe I was just using the wrong tool.

I next asked ChatGPT to change it to a web form. Lola, my name for my ChatGPT app, came back with the code shown here. (Figure 17.6)

Figure 17.6: Lola Version 2

```
using System;

class Program
{
   static void Main()
   {
      string question = "Which planet is known as the Red Planet?";
      string[] options =
      {
         "A) Earth",
         "B) Mars",
         "C) Venus",
         "D) Jupiter"
      };

      char correctAnswer = 'B';

      Console.WriteLine(question);
      Console.WriteLine();

      foreach (var option in options)
      {
         Console.WriteLine(option);
      }

      Console.WriteLine();
      Console.Write("Enter your answer (A, B, C, or D): ");

      string input = Console.ReadLine()?.Trim().ToUpper();

      if (input == correctAnswer.ToString())
      {
         Console.WriteLine("Correct!");
      }
      else
      {
         Console.WriteLine($"Incorrect. The correct answer is {correctAnswer}.");
      }
   }
}

using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Hosting;
```

My Lola, a.k.a. ChatGPT, was close but still missed the mark. See those lines "using Microsoft.AspNetCore.Builder" and "using Microsoft.AspNetCore.HTTP", well they are not within the System namespace. Copilot, Visual Studio's AI assistant, could recognize the issue but could not suggest a resolution. Again, a dead stop unless I wanted to do some research which I really wasn't too interested in doing.

I had seen enough. Both tools could generate a reasonable GUI but fell flat when asked to write a real C# web form.

At this point I feel like I need to tell you that both of these AI engines are non-Microsoft products attempting to write Microsoft-based code. Maybe that is currently too big a challenge. But at the same time, we know that Copilot is pretty good at providing possible fixes for your coding mistakes but is in no way ready to write code from scratch.

In defense of AI generated code, I recently spoke to a long time PL/SQL programmer who swears by AI. He of course clearly qualified his statement by saying you need a really detailed specification and that he always went behind to clean the code up for performance and maintainability.

Here is what I think. I consider myself an expert in both C# and PL/SQL. I can tell you for sure that C# is much, much more complex. C# has evolved from C++ which evolved from C. C goes all the way back to the early 1970s and has been changing rapidly over the years. PL/SQL is much, much simpler and has undergone much less evolution. As such, I am not surprised that the AI generation of PL/SQL code is more mature.

So, what is the final answer? AI generated code is coming along but is probably still not where it needs to be. Where will it be in 5 or 10 years? Your guess is as good as mine, but I am sure the quality of this code is only going to improve.

Will C# programmers become obsolete? I kind of doubt it. I asked my PL/SQL friend whether he was concerned he might be replaced. He was quick to respond with an emphatic “No, because you still need to know the language very well to be able to put together a decent specification. With AI the code is just put together much faster.”

What is my walk-away on AI generated code? You still need someone to guide the process. Will that be the case in 10-15 years? Nobody knows for sure, but you can bet that application requirements are going to outpace AI’s ability to meet them so stay tuned. The past 30-40 years have seen incredible advancements in both application requirements and our ability to write code which meets those requirements. I expect the growth curve will continue to accelerate and that AI generated code, and maybe even requirements, will continue to require human intervention.

Okay, that concludes Section 3 wherein we took a look at some real-world examples putting to use the fundamentals we learned in Section 2. We now move onto Section 4 wherein we learn about methods for storing and manipulating data outside our C# application. Should be great fun.